

Report Esperimenti

Francesca Orrù

1 Addestramento dei modelli

In questa sezione vengono spiegate le modalità di addestramento dei modelli utilizzati per gli esperimenti, compresa la baseline utilizzata. Ognuna di queste famiglie di modelli presentava le proprie criticità riguardanti la diversità nella distribuzione, quindi ho cercato di gestirle individualmente, partendo da delle funzioni comuni ma creandone altre ad hoc in base alla famiglia.

1.1 KNN

Il problema dei KNN risiedeva nel fatto che, per la natura del modello stesso, fosse complicato addestrarne tanti mantenendo comunque un certo livello di diversità nelle previsioni. Un suggerimento è stato quello di utilizzare dei campioni di features diversi per ogni modello. A questo scopo, ho ritenuto conveniente creare direttamente una classe knn che funzionasse come il KNeighborClassifier di scikit-learn ma che introducesse il campionamento delle features nel fit e nei predict in maniera automatica, se definito da un parametro features. Questo mi ha permesso di addestrare 1000 modelli relativamente diversi (??), includendo un campionamento casuale delle features per ognuno, senza andare a modificare le misure già create affinché gestissero internamente i predict sui campioni.

1.2 Rule Tree

Per quanto riguarda i Rule Tree, il suggerimento è stato quello di lavorare sui base stumps che compongono l'albero. A questo scopo ho creato una funzione che, gestendo il random state, mi creasse degli alberi che, per ogni combinazione di parametri, mi creasse degli alberi i quali split venissero scelti randomicamente il 60% delle volte e seguendo il best split il restante 40%. Il risultato può essere visto in ??.

1.3 Regressori Logistici

Per i regressori logistici si è optato per l'introduzione di rumore randomico post fit sui vettori dei coefficienti, introdotto con una distribuzione normale con $\text{std} = 0.5$. In aggiunta, si è percorsa la stessa strada dei KNN, aggiungendo un campionamento casuale delle features.

1.4 Baseline

Per quanto riguarda la baseline, ho addestrato quattro modelli diversi, che possono essere ritenuti lo stato d'arte al momento per quanto riguarda la performance. Ad ogni modo, sebbene tutti abbiano ottenuto performance sopra la media, si possono definire alla pari dei migliori modelli addestrati tra i knn, Rule Tree e Regressori Logistici, probabilmente perché il dataframe utilizzato (german credit) non è poi così grande, quindi i modelli così complessi tendono comunque ad andare in overfitting se non viene applicata una regolarizzazione

adeguata insieme al tuning degli altri parametri. Ciò nonostante, tendenzialmente gestiscono meglio la disparità tra classi.

Ho deciso di prendere il migliore tra i quattro in termini di f1 score, perché ho ritenuto che non fosse necessario che la baseline fosse nettamente superiore ai modelli interpretabili, essendo pertanto sufficiente un paragone tra i modelli più semplici e quelli più sofisticati. Mi sono basata sull'f1 perché, essendo le classi del df molto sbilanciate, risulta essere una metrica più affidabile dell'accuracy.

modello	accuracy	precision	recall	f1	dem parity	f1 robustness
Best f1 (RTC)	0.714286	0.517857	0.690476	0.591837	0.292063	0.926292
CatBoost	0.671429	0.469697	0.738095	0.574074	0.180952	0.944984
RandomForest	0.664286	0.460317	0.690476	0.552381	0.180952	0.941742
XGBoost	0.585714	0.404762	0.809524	0.539683	0.116279	0.969738
LGBM	0.671429	0.462963	0.595238	0.520833	0.277778	0.941518

Table 1: Paragone tra i 4 modelli da cui è stata ricavata la baseline e il migliore tra i modelli addestrati e da cui si dovrà filtrare il Rashomon Set.

2 Struttura del lavoro

L'analisi condotta in questo report è strutturata in questo modo:

- **Studio della correlazione tra metriche:** l'intento è quello di valutare quali tipi di correlazione ci siano tra le metriche calcolate per la definizione del RS e come variano queste correlazioni se valutate all'interno del RS stesso. Ci sono correlazioni particolari tra determinate metriche? Come variano se le inseriamo in un contesto che richiede una certa soglia di complessità, performance o equità dei modelli? Riusciremo ad ottenere dei modelli accurati, robusti, equi e non troppo complessi?
- **definizione Rashomon set:** determinazione e primo studio del Rashomon set, prima per ogni famiglia, poi per tutti i modelli. Lo scopo è lo studio del Rashomon Ratio al variare del livello di tolleranza, al quale si aggiunge una prima analisi delle altre metriche scalari all'interno del RS stesso, per capire come si presentano mediamente al suo interno.
- **Clustering del RS:** qua si vorrebbe provare a definire dei cluster di fairness, performance o complessità tra i modelli del RS, per capire se, per esempio, si possa individuare un cluster di modelli ben performanti con una fairness sopra la media, o con una complessità più controllata.
- **Analisi approfondita dei cluster:** qua si entra specificatamente nel contesto dell'equivalenza tra modelli. L'idea è quella di paragonare tutte le possibili coppie di modelli attraverso l'uso delle metriche di similarità e considerarne una media; stessa cosa con tutte le altre misure possibili, anche quelle non comparative. Si potrebbe inoltre considerare una coppia di modelli in particolare e farne un'analisi approfondita sfruttando tutte

le metriche possibili. Il punto critico qui sarebbe la definizione di un threshold che definisca quando due modelli siano equivalenti o meno.

Considerando che l'ottenimento del RS e i successivi step per la definizione del gruppo finale di modelli da approfondire sono di per sé operazioni di model selection, tutte le analisi inerenti saranno fatte su un validation set, mentre l'analisi finale sull'equivalenza dei modelli scelti verrà fatta su un test set finale.

3 Studio correlazione e distribuzione delle metriche

In questa sezione, lo scopo è quello di valutare come le metriche calcolate sui modelli sono distribuite e correlate: questo ci permette di osservare se i modelli sono ben diversificati e come, per esempio, la complessità di un modello può influenzare la performance o la robustezza.

Le metriche in questione sono state suddivise in 5 macro-gruppi:

- **Performance:** Accuracy, Precision, Recall, f1 score;
- **Fairness:** Equalized Odds, Conditional Use Accuracy Equality, Demographic Parity, Equal Opportunity, Predictive Equality, Predictive Parity, Negative Predictive Parity;
- **Fairness Robustness:** Demographic Parity Robustness, Equal Opportunity Robustness, Predictive Equality Robustness, Predictive Parity Robustness, Negative Predictive Parity Robustness, Equalized Odds Robustness;
- **Performance Robustness:** Accuracy Robustness, Precision Robustness, Recall Robustness, f1 Robustness;
- **Complexity:** number of neighbors (KNN), Coefficients Magnitude (LR), Tree Depth (RTC), number of rules (RTC), number of nodes (RTC);

A scopo di visualizzazione, ho pensato che fosse più informativo considerare una variabile rappresentativa di ogni macro-gruppo. Per questo motivo, ho applicato una PCA (dove necessaria) per ogni gruppo e ottenuto una variabile che ne riassume la relazione. Chiaramente le analisi sui valori esatti verranno fatte considerando ogni singola metrica.

Sempre con il solo scopo di facilitare la visualizzazione, ho applicato del preprocessing alle metriche di fairness.

Le misure di fairness calcolate hanno generalmente un intervallo (0,1) o (-1,1): più si avvicinano allo 0, più il modello è equo. Per comodità ho deciso di sottrarre il loro valore assoluto a 1, in modo tale che la metrica rappresentativa della fairness aumentasse con l'aumentare dell'equità;

$$inverted_metric = 1 - |x| \tag{1}$$

Essendo x una delle metriche di equità all'interno del gruppo Fairness. Per il resto delle metriche che avevano bisogno, ho applicato una trasformazione logaritmica lungo gli assi dei grafici.

REMINDER: considerando che il concetto di complessità varia a seconda della famiglia di modelli, lo studio di questa viene fatto solo nelle analisi per modelli della stessa famiglia.

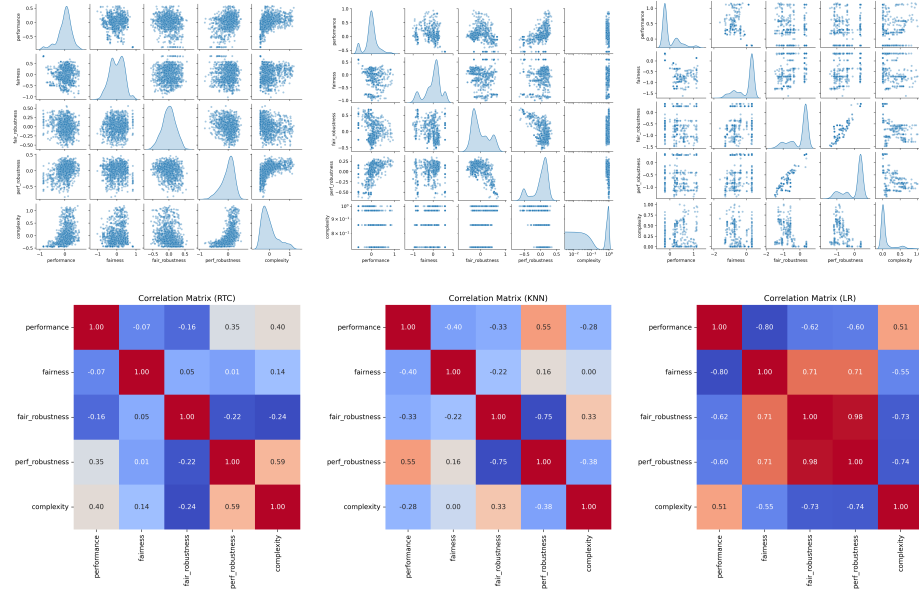


Figure 1: Correlazioni tra le macro-metriche calcolate sui modelli e le rispettive distribuzioni (sopra) e valori delle correlazioni (sotto), per ogni famiglia di modelli.

La **Figure 1** ci mostra come sono distribuiti i modelli per ogni coppia di metriche, con lo scopo di valutare come queste si influenzano a vicenda e ci offre un primo spunto superficiale su come possano essere distribuite le misure all'interno di ogni gruppo. Per completezza, vengono mostrate anche le matrici di correlazione per le misure, in modo tale da quantificare le relazioni osservate negli scatter plot.

Per gli RTC la situazione è molto promettente: infatti notiamo che i modelli, sebbene in alcuni casi concentrati in determinate zone, sono distribuiti lungo tutta la superficie. Questo vuol dire che, per esempio, in un RS con valori alti di performance avremo alte probabilità di trovare modelli più semplici o più equi. Le metriche più correlate in questo caso sono la complexity e la robustezza in performance, ma anche qui, per come sono distribuiti i modelli, non sarà difficile trovare modelli sia semplici che robusti in un eventuale RS.

Per quanto riguarda i KNN, ciò che incuriosisce è la relazione tra la performance e la complessità dei modelli: la correlazione non è alta, ma dagli scatter

plot si evince che, in un eventuale RS, non avremo troppe vie di mezzo sulla complessità, considerando i gap tra i valori. Per quanto riguarda la relazione tra equità e performance, possiamo notare che, per valori alti di performance, c'è un piccolo cluster poco denso di modelli con equità medio alta che potremo ritrovarci nel RS, anche se la percentuale sarà sicuramente più bassa che per i RTC.

Infine, i LR risultano essere i più problematici in termini di sparsità: ci basti guardare prima gli scatter plot e poi gli elevati livelli di correlazione per ogni combinazione di metriche. Banalmente, in un RS basato sulla performance, faremo fatica a trovare modelli semplici, equi o robusti. Allo stesso momento, un RS di modelli con performance ed equità o semplicità elevate sarà praticamente vuoto, come si evincerà nelle sezioni successive. Ciò nonostante, se guardiamo la **Figure 2** notiamo che in generale, la percentuale dei LR che si avvicina all'equità è quasi sempre più elevata delle altre famiglie, ma in un eventuale RS sulla performance verrebbero scartati.

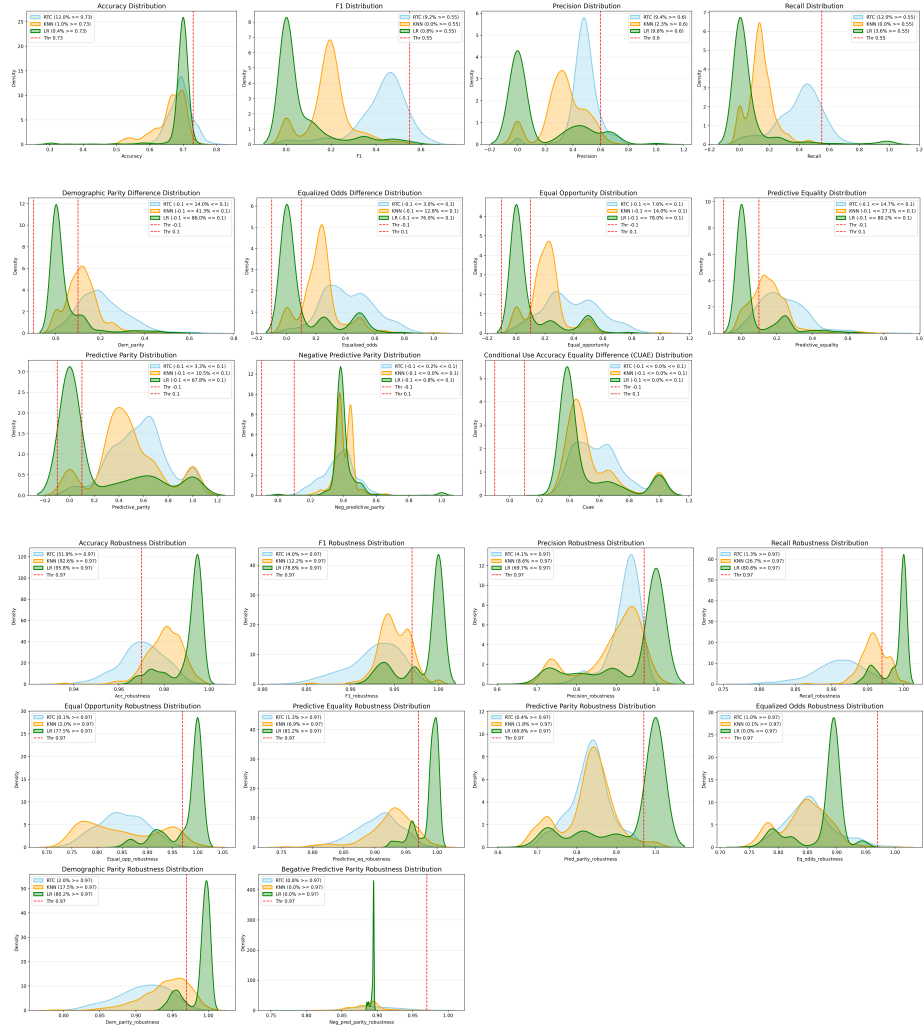


Figure 2: Analisi delle distribuzioni delle metriche, famiglia per famiglia.

La **Figure 2** invece ci mostra un paragone delle famiglie sulle metriche calcolate. In generale, possiamo affermare che mediamente i RTC performano meglio dei KNN e LR: per valori elevati di performance, ci aspetteremo di avere una percentuale maggiore di RTC rispetto ai KNN o ai LR, che sono quelli meno presenti di tutti sopra un certo livello di performance.

Al contrario però, come detto prima, i LR hanno la più alta concentrazione di modelli equi per quasi tutte le metriche di equità, mentre i RTC sembrano essere i più penalizzati sotto questo aspetto. Risulta anche, che la maggior parte dei modelli più robusti, sia per equità che per performance, siano sempre i LR.

Più in generale, possiamo vedere dai grafici di performance e fairness come i modelli faticino a bilanciare bene le classi nelle loro previsioni mentre, dai

range di robustezza abbastanza ristretti su valori medio-alti, si evince che tutti i modelli siano abbastanza robusti sia per equità che per performance.

4 Studio del Rashomon Set

Nella costruzione dei RS, ci sono due questioni principali da affrontare: il livello di tolleranza ϵ e le metriche su cui ricavarlo.

Per ragioni riguardanti la comodità, ho ritenuto adeguato definire il primo RS basandomi solo sulla performance, scegliendo l'f1 perché la più veritiera in questo df, per poi proseguire nelle sezioni successive con il restringimento del set attraverso un processo di clustering sul resto delle metriche. Questa scelta è dovuta al fatto che altrimenti avremmo dovuto studiare e definire un ϵ per ogni gruppo di metriche utilizzate, se non per singola metrica, il che sarebbe stato gravoso. Ad ogni modo, è bene specificare che, volendo, il RS potrebbe essere definito rispetto a più threshold, oltre a quello sulla f1 (f1 + demographic parity per fare un esempio).

Per quanto riguarda la definizione del livello di tolleranza, la letteratura non offre euristiche dirette su come sceglierlo, ma solitamente si aggira intorno a valori del 5%-20% massimo del valore di performance della baseline di riferimento. Questo probabilmente è dovuto al fatto che il suo valore dipenda anche dal contesto in cui si lavora e dal livello di rischio della performance dei modelli che si è disposti a sostenere. Personalmente, ho svolto prima uno studio della variazione del RR al variare di ϵ per capire quanto ci si dovesse spingere oltre per arrivare a un RS accettabile in termini di grandezza e di proporzioni sulle famiglie di modelli osservate (**Figure 3**). Da qui si nota come sia necessario, nel nostro caso, applicare valori di tolleranza di almeno 0.15 per poter avere una minima rappresentanza di tutte le famiglie all'interno di un RS generale scelto sull'f1, mentre basta uno 0.1 di tolleranza per avere un RS solo di RTC.

La **Figure 4** mostra bene questo trade-off e mi porta a voler proseguire con una tolleranza dello 0.2 per ragioni di studio, nonostante si veda che i KNN siano nettamente meno rappresentati rispetto ai RTC e LR.

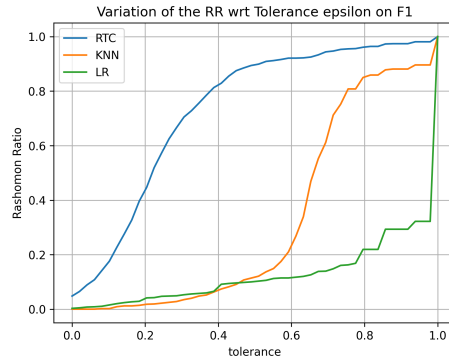


Figure 3: Studio della variazione del RR rispetto all'aumento del livello di tolleranza ϵ sull'f1 score.

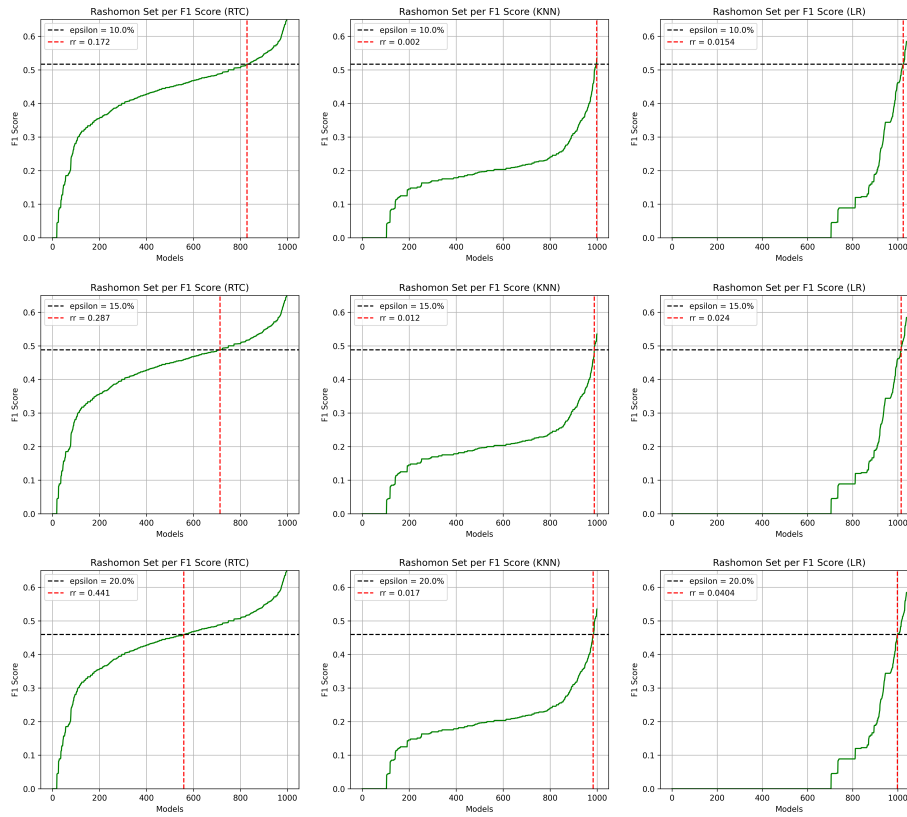


Figure 4: Visualizzazione del RR considerando livelli di tolleranza 0.1, 0.15 e 0.2 sull'f1. Le linee nera e rossa stabiliscono la soglia del RS.

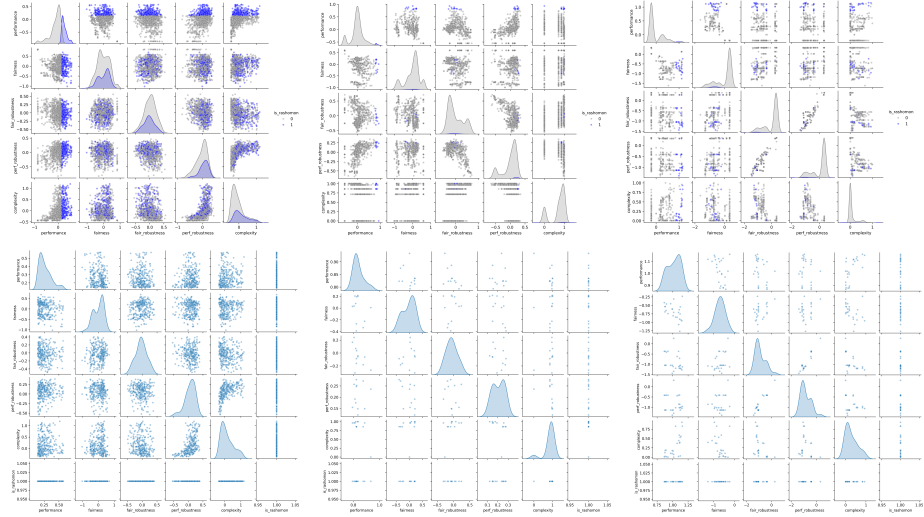


Figure 5: Correlazioni tra le macro-metriche calcolate sui modelli del RS e le rispettive distribuzioni per ogni famiglia di modelli e per il RS totale. In alto osservate all'interno dell'intero set di modelli, in basso osservate da sole per comprendere meglio la distribuzione.

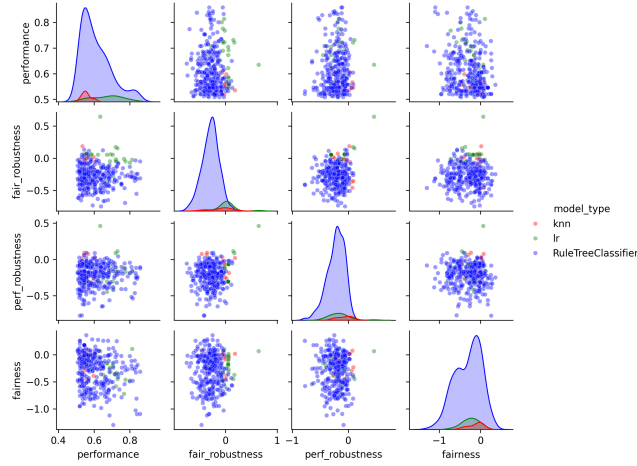


Figure 6: La figura mostra il RS inter-family che si divide nei gruppi delle tre famiglie osservate.

5 Studio dell'equivalenza tra modelli interni al RS

Come si è visto nelle sezioni precedenti, il RS costituisce solo una prima selezione di modelli, definiti equivalenti perché superano un certo livello di performance

che ci permette di non fare distinzioni in tal senso. Ad ogni modo, due o più modelli possono essere considerati equivalenti non solo se hanno la stessa performance, ma anche se hanno lo stesso modo di ragionare, attribuire importanza alle variabili, interpretabilità, robustezza alla variazione dei dati ed equità nelle scelte per quanto riguarda una determinata variabile sensibile. Questo è provato dal fatto che, all'interno del RS, abbiamo un'alta varianza nella complessità, robustezza, equità e, come ci si aspetta di vedere successivamente, anche in tutti gli altri aspetti sopra citati.

Possiamo definire tali aspetti sull'equivalenza formalmente in questo modo:

- **Logico:** come i modelli si somigliano nelle previsioni che fanno. Sbagliano sugli stessi esempi? Hanno la stessa confidence sulle previsioni in caso di discordanza/concordanza? Se entrambi hanno torto/ragione, quanto sono convinti sulla ground truth?
- **Importanza delle variabili:** come i modelli si somigliano sull'importanza che attribuiscono alle diverse variabili a disposizione. Può essere calcolata specificatamente per famiglia, oppure può essere agnostica al tipo di modello (usando SHAP nel nostro caso);
- **Complessità:** Quanto differisce il grado di difficoltà con cui l'utente può interpretare il modello?
- **Fairness:** i modelli trattano allo stesso modo le classi di una variabile sensibile?
- **Robustezza nella fairness:**
- **Robustezza nella performance:**

Lo scopo di questa analisi è quello di studiare sotto quali di questi aspetti i modelli sono da considerarsi equivalenti e, perciò, quanto si dicono equivalenti.

Lo studio parte da delle matrici di errore calcolate sulle principali metriche scalari già viste, più la differenza tra previsioni. Ognuna di queste matrici è simmetrica, ha come righe e colonne i modelli del RS e come valori le differenze assolute (per le metriche scalari) e le distanze di Hamming normalizzate (per i vettori delle previsioni) per ogni coppia di modelli. La distanza di Hamming è stata scelta in quanto gestisce anche il multiclasse, a differenza della Jaccard. Per questo motivo, ho deciso di adottare il Consensus Clustering:

- **Clustering matrici di distanza:** prendiamo le matrici di distanza calcolate su ogni metrica e applichiamo un algoritmo di clustering, per avere una prima idea di come i modelli si raggruppano nel limite della metrica in questione. alla fine abbiamo m clustering $C^{(m)}$, uno per metrica;
- **Calcolo matrici di co-associazione:** partendo dai risultati dei clustering, si calcolano m matrici di co-associazione che indicano se due modelli

i, j si trovano nello stesso cluster:

$$A_{i,j}^{(m)} = \begin{cases} 1 & \text{se } i, j \text{ sono nello stesso cluster} \\ 0 & \text{altrimenti} \end{cases}$$

- **Calcolo matrice di consenso:** il nostro scopo è sapere quanto ogni coppia di modelli è equivalente e la matrice di consenso ci serve proprio per capire questo: infatti, rappresenta la somma pesata delle matrici di co-associazione e ci quantifica la co-presenza di ogni coppia di modelli in uno stesso cluster:

$$A^{cons} = \sum_m w_m A^{(m)}$$

Il **vettore pesi w** serve per fare in modo che ogni categoria di metriche abbia la stessa importanza a prescindere dalla quantità di metriche in sua rappresentanza. La somma dei suoi elementi è 0;

- **Clustering sulla matrice di consenso:** una volta ottenuta la matrice di consenso, viene applicato un altro clustering con lo scopo di selezionare dei **cluster di modelli con diversi livelli di equivalenza**. Il clustering C^{cons} viene applicato più precisamente su $1 - A^{cons}$, perché a noi serve una matrice di distanza per avere più scelta sugli algoritmi di clustering;

Scelta delle metriche Per semplificare il lavoro, per ogni categoria di metriche sono state **ignorate quelle con alta correlazione** perché ridondanti. Ho pensato all'inizio di lavorare sulle rappresentanti di categoria ottenute per **PCA** in precedenza, ma chiaramente lo studio approfondito delle metriche sarebbe risultato **incoerente**. Queste sono le metriche che ho tenuto per l'analisi:

- **Fairness:** Equalized Odds, Demographic Parity, Predictive Parity, Negative Predictive Parity;
- **Performance Robustness:** Accuracy Robustness, F1 Robustness, Precision Robustness, Recall Robustness;
- **Fairness Robustness:** Demographic Parity Robustness, Equal Opportunity Robustness, Predictive Parity Robustness, Negative Predictive Parity Robustness, Equalized Odds Robustness;
- **Previsioni:** vettori delle previsioni sul validation set su cui calcolerò la Hamming.

Ho deliberatamente scelto di considerare le metriche di fairness, complexity e robustness all'interno dello studio sull'equivalenza e non preventivamente per la definizione del RS, ma avrei potuto, facendo però una selezione più rigida e rischiando di non avere modelli da studiare per determinate famiglie o in generale.

Scelta delle misure di distanza Le due misure che ho utilizzato sono:
 $MAE = \frac{1}{n} \sum_n |a_n - b_n| = |a - b|$ (per scalari)

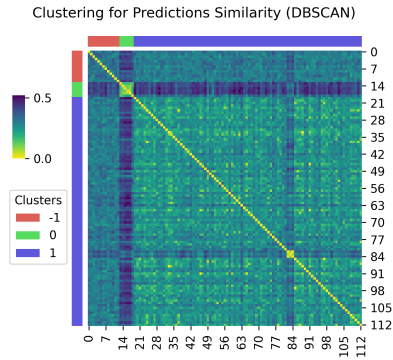
$$d_H(a, b) = \frac{1}{n} \sum_n \mathbf{1}(a_n \neq b_n)$$

Il MAE diventa una differenza assoluta se usato su metriche scalari come ho fatto io e la **Hamming Distance è utile per vettori di previsioni multiclasse** perché, diversamente dalla Jaccard classica, non accetta solo vettori binari.

Perché lavorare sulle distanze e non sulle similarità All'inizio ho pensato di lavorare direttamente sulle similarità per una questione interpretativa, ma poi mi sono resa conto che mi avrebbe **precluso la scelta di diversi algoritmi** di clustering che, a differenza di un Clustering Spettrale, lavorano sulle distanze e non sulle affinità.

Scelta degli algoritmi di clustering Considerando il mio interesse nel valutare anche la presenza di outliers nei clustering iniziali, ho puntato su algoritmi di densità come il **DBSCAN, HDBSCAN e OPTICS**. Per il momento ho usato solo il DBSCAN rendendomi conto che i cluster non sono efficienti. Per il clustering finale invece, il mio scopo era quello di valutare a che livello di equivalenza (o nel nostro caso distanza) i cluster si assottigliavano: motivo per cui ho scelto un clustering gerarchico per visualizzare il dendrogramma e scegliere poi i cluster in base al livello della distanza.

5.1 Risultati



cluster	avg dist	n LR	n RTC
0	0.1429	6	-
1	0.2569	3	91
out	0.3205	-	13

Figure 7: Studio dei clustering sulla matrice di distanza per le previsioni dei modelli.

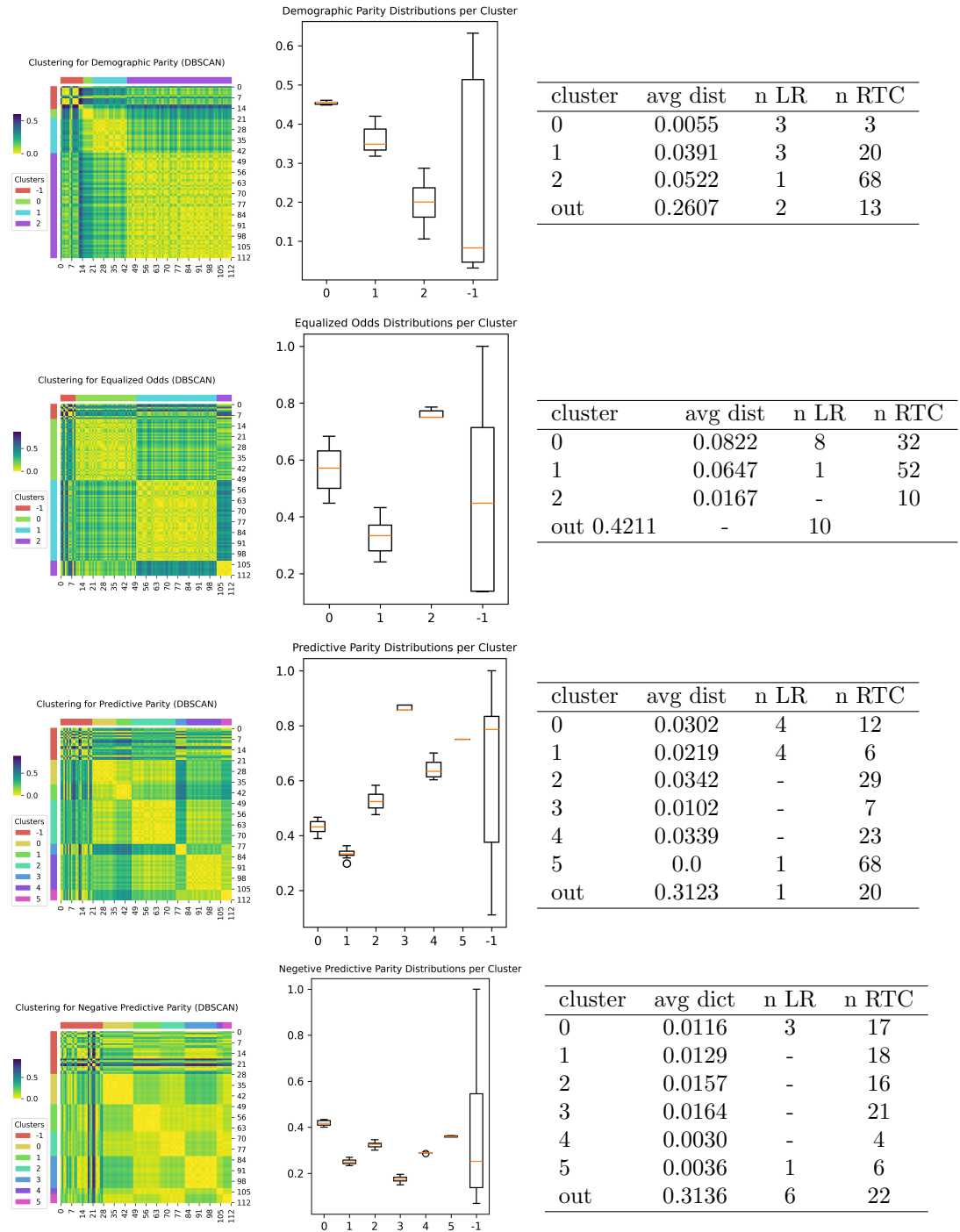


Figure 8: Studio dei clustering sulle matrici di distanza per le metriche di fairness

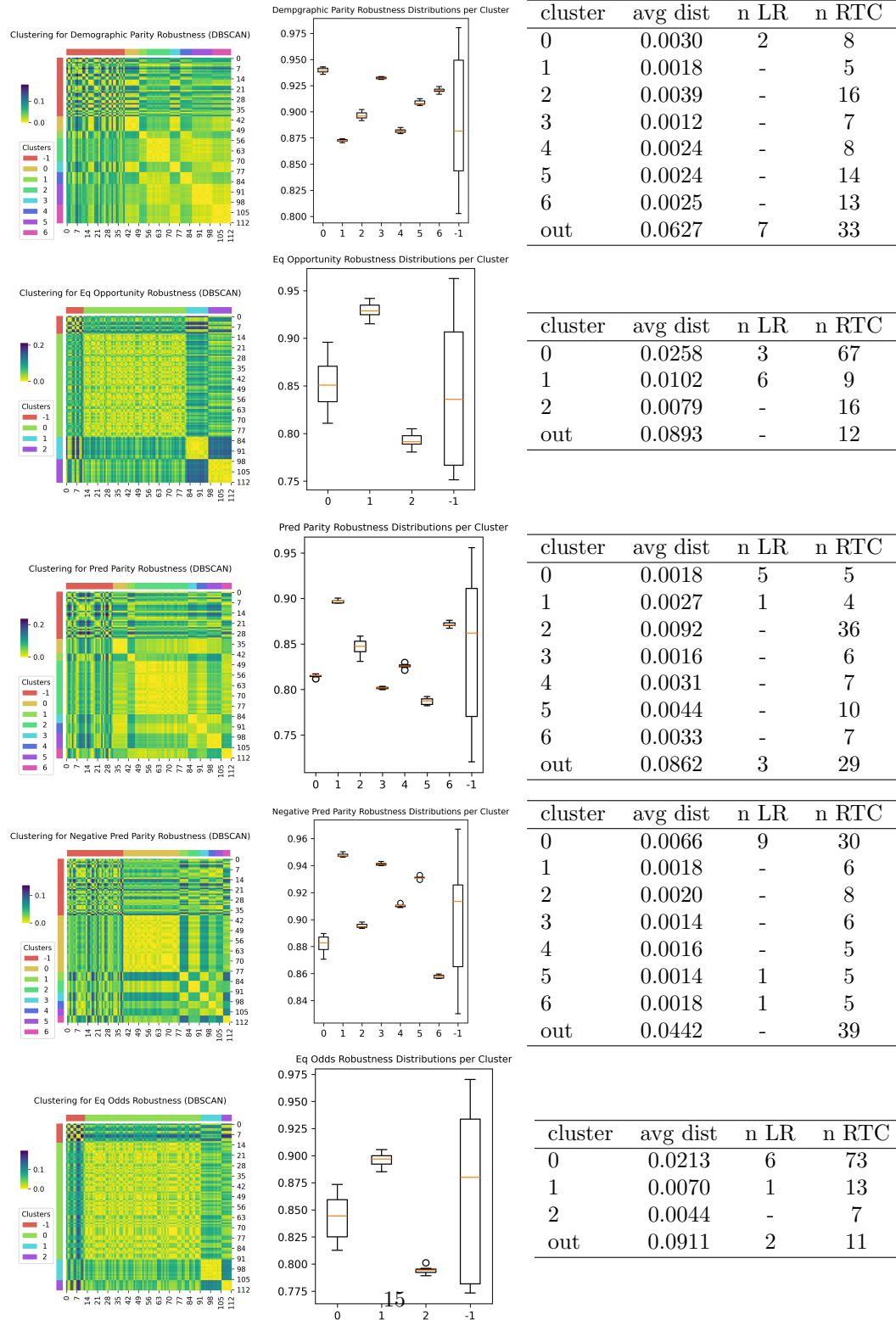


Figure 9: Studio dei clustering sulle matrici di distanza per le metriche di fairness robustness.

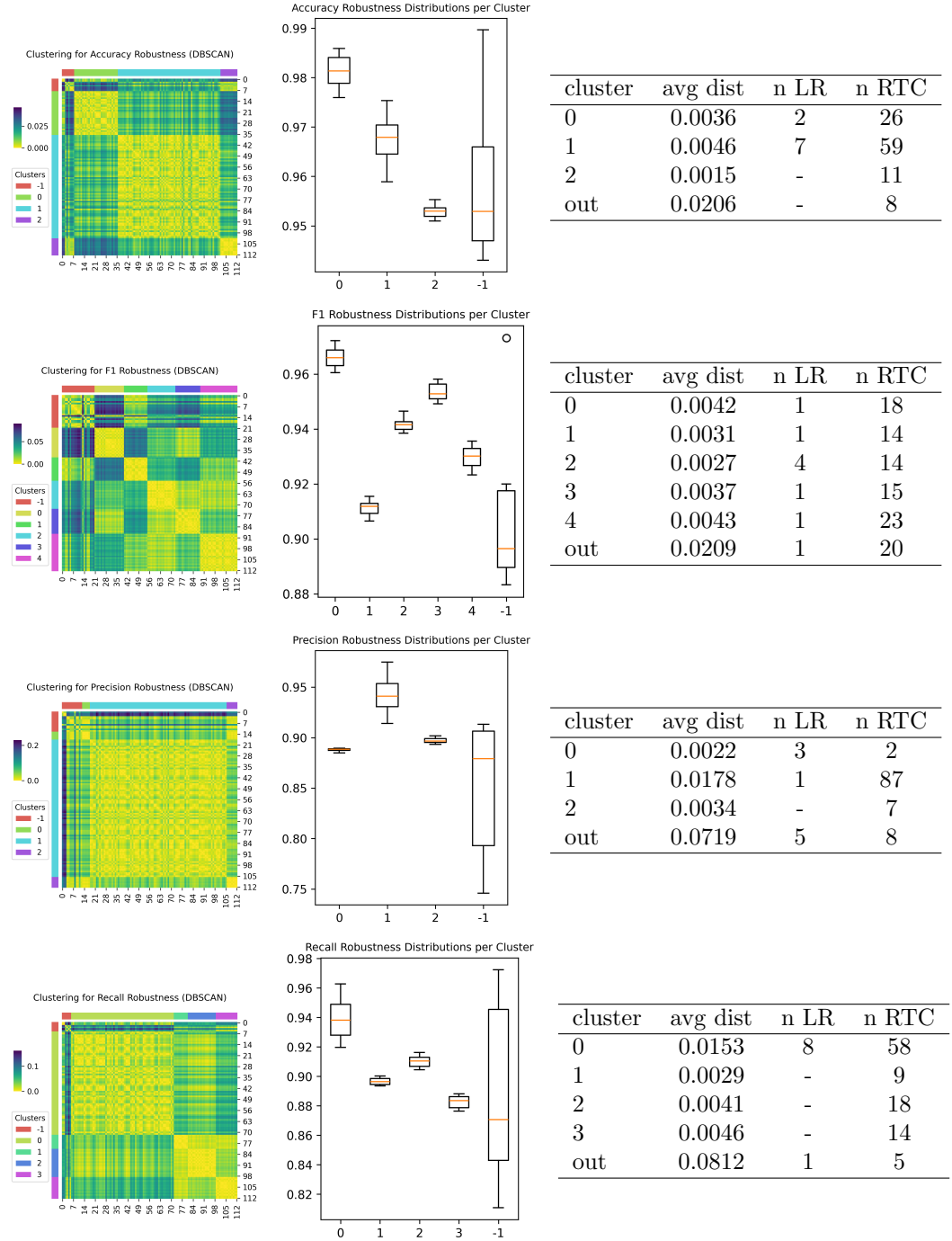


Figure 10: Studio dei clustering sulle matrici di distanza per le metriche di performance robustness.

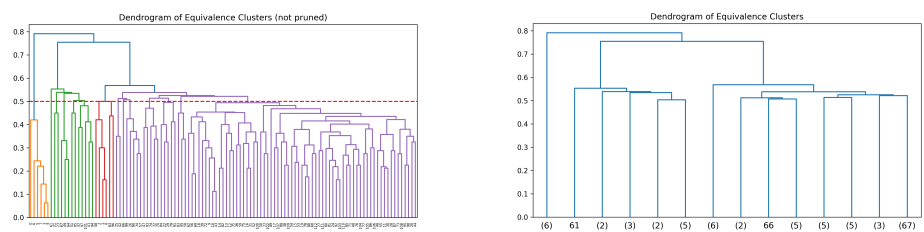


Figure 11: Dendrogramma del clustering finale.